



GOVERNO DO ESTADO DO RIO DE JANEIRO
SECRETARIA DE ESTADO DE CIÊNCIA E TECNOLOGIA
FUNDAÇÃO DE APOIO À ESCOLA TÉCNICA
CENTRO DE EDUCAÇÃO PROFISSIONAL EM TECNOLOGIA DA INFORMAÇÃO
FACULDADE DE EDUCAÇÃO TECNOLÓGICA DO ESTADO DO RIO DE JANEIRO
FAETERJ/PETRÓPOLIS

***Sistema para a Gestão de Professores Vinculados à
Instituição e Montagem da Grade de Horário do
Curso de Tecnologia da Informação e Comunicação
da Faeterj-Petrópolis***

João Pedro Moura Ferreira

Petrópolis - RJ

2025, Fevereiro

João Pedro Moura Ferreira

***Sistema para a Gestão de Professores Vinculados à
Instituição e Montagem da Grade de Horário do
Curso de Tecnologia da Informação e Comunicação
da Faeterj-Petrópolis***

Trabalho de Conclusão de Curso apresentado à
Coordenadoria do Curso de Tecnólogo em Tec-
nologia da Informação e da Comunicação da
Faculdade de Educação Tecnológica do Estado
do Rio de Janeiro Faeterj/Petrópolis, como re-
quisito parcial para obtenção do título de Tec-
nólogo em Tecnologia da Informação e da Co-
municação.

Orientador:

Leonardo da Silva Gomes

Petrópolis - RJ

2025, Fevereiro

Folha de Aprovação

Trabalho de Conclusão de Curso sob o título “*Sistema para a Gestão de Professores Vinculados à Instituição e Montagem da Grade de Horário do Curso de Tecnologia da Informação e Comunicação da Faeterj-Petrópolis*”, defendido por João Pedro Moura Ferreira e aprovado em dez de fevereiro de dois mil e vinte e cinco, em Petrópolis - RJ, pela banca examinadora constituída pelos professores:

Prof. Leonardo da Silva Gomes
Orientador

Prof. Chessman Kennedy Faria Correa
Faculdade de Educação Tecnológica do Estado
do Rio de Janeiro Faeterj/Petrópolis

Prof. Amarildo Micoski Luz
Faculdade de Educação Tecnológica do Estado
do Rio de Janeiro Faeterj/Petrópolis

Declaração de Autor

Declaro, para fins de pesquisa acadêmica, didática e tecnico-científica, que o presente Trabalho de Conclusão de Curso pode ser parcial ou totalmente utilizado desde que se faça referência à fonte e aos autores.

João Pedro Moura Ferreira

Petrópolis, em dez de fevereiro de dois mil e vinte e cinco

Dedicatória

Dedico este trabalho aos meus pais Marcos e Sandra, aos meus irmãos Marcos Paulo e Natiele, aos meus Sobrinhos Thomas, Thiago e Maria Luiza, aos meus cunhados Luiz e Vanessa, a todos os meus amigos e familiares, em especial a meus avós, minha tia Renata e minha prima Priscila, que já nos deixaram.

Agradecimentos

Agradeço primeiramente a Deus, por me dar a vida, proporcionar as condições necessárias para finalizar este trabalho e, principalmente, por me fazer cruzar, ao longo da minha jornada, com todas as pessoas citadas a seguir.

Aos meus pais, Marcos e Sandra, que sempre estiveram ao meu lado, me apoiando incondicionalmente e guiando-me nos momentos desafiadores. A sabedoria e o amor que me transmitem são fundamentais para o meu crescimento. Sou eternamente grato por cada ensinamento e por cada palavra de encorajamento, além de estarem sempre presentes em minha jornada.

Aos meus irmãos, Marcos Paulo e Natiele, agradeço pelos conselhos e pelo ótimo convívio. Aos meus cunhados, Vanessa, que me ajudou muito neste trabalho, e Luiz, e aos meus sobrinhos, Thomas, Thiago e Maria Luiza, por toda a alegria proporcionada e os momentos juntos.

Ao professor Leonardo, que me orientou e ajudou a concluir este trabalho, aos meus amigos, Gustavo, Isaque, José, Nathanael, Pedro, Thales e Victor, por todos os momentos que passamos juntos, seja jogando ou apenas conversando.

Por fim, agradeço à Esteira de Serviços Digitais e ao Time de Inovação, meus antigo e atual time no trabalho, pelos ensinamentos e pelo convívio diário.

Sem o apoio de todos vocês, não seria capaz de concluir este trabalho.

Epígrafe

"Está tudo bem em chorar. Está tudo bem em fugir.
Apenas não desista."

(Gotouge, Koyoharu; Kimetsu no Yaiba 2019)

Resumo

Este trabalho apresenta uma abordagem sobre os problemas relacionados à confecção da grade de horários para o curso Tecnólogo em Tecnologia da Informação e da Comunicação da Faculdade de Educação Tecnológica do Estado do Rio de Janeiro Faeterj/Petrópolis. O problema consiste na confecção da grade de horários do curso, que deve respeitar diversas restrições, como a quantidade de aulas semanal de cada disciplina, a alocação de horários para os professores de forma que o mesmo esteja lecionando somente uma disciplina por horário e que mais de um professor não esteja associado a uma mesma disciplina. Tal processo, geralmente feito de forma manual, demanda um elevado esforço, repetido sempre que necessário. A automatização do processo de desenvolvimento da grade de horário, se faz necessária mediante ao grande nível de complexidade e o tempo gasto na elaboração das grades de horário. O procedimento para automatização consiste em relacionar os dados daquele amplo conjunto de restrições, e gerar iterativamente um horário para cada turma identificando o professor e disciplina.

Palavra-chave: grade de horário; disciplina; matéria; alocação; automatização;

Abstract

This paper presents an approach to the problems related to creating the class schedule for the course Tecnologia da Informação e da Comunicação da Faculdade de Educação Tecnológica do Estado do Rio de Janeiro Faeterj/Petrópolis. The issue lies in designing the schedule, which must comply with several constraints, such as the required number of weekly classes for each subject, assigning time slots for professors so that each teaches only one subject per period, and ensuring that no more than one professor is assigned to the same subject. Typically done manually, this process is time-consuming and requires considerable effort, being repeated whenever necessary. Given the high level of complexity and the time spent on schedule creation, automating this process becomes essential. The automation procedure involves analyzing the broad set of constraints and iteratively generating a schedule for each class, identifying both the professor and the subject.

KEYWORD: class schedule; classes; subject; automation; assigning;

Sumário

Lista de Figuras

Lista de Tabelas

1	Introdução	p. 13
2	Problema de Quadro de Horário	p. 14
2.1	Classificação de Problemas	p. 15
3	Algoritmos Exatos e Meta-Heurísticos	p. 17
3.1	Algoritmos Exatos	p. 17
3.1.1	Branch and Bound	p. 17
3.1.2	Relax and Cut	p. 18
3.2	Algoritmos Meta-Heurísticos	p. 18
3.2.1	Busca Local	p. 18
3.2.2	Algoritmo Genético	p. 20
3.2.3	GRASP(<i>Greedy Randomized Adaptative Search Procedure</i>)	p. 22
4	Definição do Problema e Proposta para solução do problema	p. 24
5	Solução	p. 27
5.1	Resultados	p. 27
5.2	Caso de Uso	p. 30
5.2.1	Diagrama de Caso de Uso	p. 30
5.2.2	Especificações dos Casos de Uso	p. 30

5.3	Descrição das Classes	p. 39
5.4	Diagrama de Entidade e Relacionamentos.	p. 40
5.5	Tecnologias Utilizadas	p. 40
5.5.1	Windows Forms .NET	p. 40
5.5.2	Visual C#	p. 41
5.5.3	Microsoft Visual Studio Community 2022	p. 41
5.5.4	Microsoft SQL Server 2017 Express	p. 41
6	Conclusão	p. 42
	Referências	p. 44

Lista de Figuras

2.1	Diagrama de Euler para P, NP, NP-Completo e NP-Difícil	Fonte: Libório, Felipe [1]	p. 16
3.1	Algoritmo básico da Busca Tabu apresentado por Souza[16]		p. 19
3.2	Estrutura básica de um Algoritmo Genético simples.	Fonte: Miranda [17]	p. 20
3.3	<i>Crossover</i> de indivíduos em um AG simples.	Fonte: RODRIGUES, F.L. et al.[18]	p. 21
3.4	Algoritmo básico GRASP.	Fonte: Fredo e Brito[23]	p. 22
4.1	Montagem da Lista de Candidatos.	Fonte: Ferreira, João Pedro Moura	p. 25
4.2	Código Final.	Fonte: Ferreira, João Pedro Moura	p. 26
5.1	Grade de Horário do Primeiro Período.	Fonte: Ferreira, João Pedro Moura	p. 27
5.2	Grade de Horário do Segundo Período.	Fonte: Ferreira, João Pedro Moura	p. 28
5.3	Grade de Horário do Terceiro Período.	Fonte: Ferreira, João Pedro Moura	p. 28
5.4	Grade de Horário do Quarto Período.	Fonte: Ferreira, João Pedro Moura	p. 28
5.5	Grade de Horário do Quinto Período.	Fonte: Ferreira, João Pedro Moura	p. 29
5.6	Diagrama de Caso de Uso.	Fonte: Ferreira, João Pedro Moura	p. 30
5.7	Diagrama de Classes.	Fonte: Ferreira, João Pedro Moura	p. 39
5.8	Diagrama de Entidade e Relacionamento.	Fonte: Ferreira, João Pedro Moura	p. 40

Lista de Tabelas

1 Introdução

A construção da grade de horários é um desafio enfrentado todo início de semestre, e é nesse instante que a faculdade se depara com a necessidade de ofertar, para os alunos, todas as disciplinas do curso, com professores previamente alocados.

Na fase de montagem da grade, devem ser levadas em consideração algumas restrições, tais como disciplinas não terem mais de um professor alocado a elas, ou disciplinas ministradas por um mesmo professor coincidirem o horário.

Por conta de todos os problemas envolvendo a construção das grades de horário, diversos estudos voltados para o problema de automatização de processos para construção de grade de horários foram realizados.

Tal problema é tratado como, os chamados problemas de otimização combinatória que, segundo Libório[1], consistem em encontrar uma solução que maximize ou minimize uma função objetivo cujo domínio é um conjunto finito de objetos. Otimização combinatória é um subconjunto da otimização matemática que está relacionado à pesquisa operacional, algoritmos e complexidade computacional. Uma solução ótima pode ser obtida através de uma simples enumeração, porém, devido ao número extremamente alto de soluções possíveis, isto acaba se tornando inviável. Nesse caso, mesmo que se utilize um supercomputador, o tempo de processamento poderia ser de várias semanas ou até anos.

Mesmo com o avanço na tecnologia, atualmente problemas que não possuem solução em tempo hábil ainda existem, outros possuem demanda de memória além das disponíveis nos computadores convencionais, além daqueles que ainda não existem uma solução conhecida, ou seja, solução essa que pode ou não existir.

Diante disso este trabalho tem como objetivo a automatização da criação da grade de horários do curso, Tecnólogo em Tecnologia da Informação e da Comunicação, ministrado na Faculdade de Educação Tecnológica do Estado do Rio de Janeiro Faeterj/Petrópolis, a proposta do mesmo visa solucionar o problema com resultado considerado satisfatório, através de um sistema com boa performance, e fácil usabilidade.

2 *Problema de Quadro de Horário*

O Problema de Quadro de Horário (*Timetabling Problem*), é bastante explorado na literatura, e de acordo com Cooper e Kingston (1995)[2] consiste em atribuir horários, professores, disciplinas e, salas a um conjunto de reuniões de modo que nenhum dos participantes tenha que comparecer em duas reuniões simultaneamente, Wren (1996)[3] descreve como o problema de colocar certos recursos, em um limitado número de *slots*.

Segundo Carvalho(2011) [4] As restrições impostas a estes problemas podem ser divididas em duas classes: rígidas e flexíveis, onde:

- Rígidas:são as que viabilizam uma solução, ou seja, quando não atendidas, a solução passa ser inviável;
- Flexíveis: medem o grau de satisfabilidade de uma dada solução viável.

Abaixo são listadas algumas restrições básicas para problemas dessa natureza e sua classificação:

- Um professor não pode lecionar mais de uma aula ao mesmo tempo (Rígida).
- Uma turma não pode ter mais de uma aula em um mesmo horário (Rígida).
- Uma turma não pode ter mais de um Professor alocado em uma mesma disciplina (Rígida).
- Alocar aulas de uma mesma disciplina de forma consecutivas, evitando aulas isoladas, ou janelas entre aulas (Flexível).

De acordo com Bardadym(1996)[5], a solução manual de um problema de *timetabling* é uma tarefa árdua e normalmente requer vários dias de trabalho. Essa tarefa, acaba se transformando num grande quebra cabeça, num pesado processo de tentativa e erro que, na primeira solução encontrada, dá-se por solucionado, mesmo que a qualidade da solução seja péssima.

O Problema de Quadro de Horário pertence à classe de problemas chamada NP-completo, ou seja, no presente momento, não se conhece nenhum método para resolvê-lo em uma quantidade de tempo razoável.

Independentemente da técnica utilizada, é importante saber e obedecer tais restrições, para evitar que no momento da geração das grades horárias, apareçam surpresas desagradáveis, como soluções inválidas e ter que refazer o processo novamente. Ao se utilizar o processo manual, refazer o processo demanda ainda mais tempo e esforço.

2.1 Classificação de Problemas

Segundo Carvalho[4], os algoritmos podem ser classificados em relação a complexidade como P, NP, NP-Completo e NP-Difícil. Onde:

- *P(Deterministic Polynomial Time)*: são aqueles que são possíveis de mostrar que existe um algoritmo, que no pior caso seja de ordem polinomial, onde a ordem polinomial de um algoritmo é uma função polinomial que limita o tempo de execução do algoritmo para resolver um problema. O algoritmo é polinomial se o tempo de execução é limitado por uma função polinomial com variável igual ao tamanho da instância do problema que o resolva.
- *NP (Nondeterministic Polynomial Time)*: são algoritmos que possuem tempo de execução com teto de $O(n^k)$, para alguma constante k , onde n consiste no número de entradas do problema.
- **NP-Completo**: consiste na classe de problemas mais difíceis entre os problemas de NP. Para que um problema se enquadre nessa classe, é necessário:
 - Mostrar que o problema está em NP;
 - Mostrar que é possível fazer uma transformação polinomial com algum problema conhecido como NP-Completo ao problema estudado.

Essa classe é dita a mais difícil no meio científico, devido ao estudo realizado por Cook[6]. A relação entre os problemas dessa classe permite dizer que se caso um problema NP-Completo seja resolvido em tempo polinomial, todos os outros poderão ser resolvidos.

- **NP-Difícil**: são compostos por problemas que atendam ao item 2 (NP-Completo) descrito anteriormente. Deste modo, todo problema NP-Completo está contido em NP-Difícil.

Porém, problemas NP-Difíceis podem não ser um problema de decisão, ou seja, logo nem todos os problemas NP-Difíceis são NP-Completo.

Sendo um problema NP, segundo Libório[1], não há solução ótima de tempo polinomial conhecida. Assim, para instâncias grandes destes problemas, busca-se usar técnicas que gerem soluções aproximadas de boa qualidade e que possam ser executadas em tempo hábil, ou seja, dificilmente terá um algoritmo exato para resolução em tempo hábil.

De acordo com Cormen, et, al.(2002)[7] se algum problema NP-completo pode ser resolvido em tempo polinomial, então todo problema em NP tem uma solução em tempo polinomial, isto é, $P = NP$. Entretanto, apesar de anos de estudo, nenhum algoritmo de tempo polinomial jamais foi descoberto para qualquer problema NP-completo.

A Figura 2.1 apresenta uma representação visual da relação entre as diferentes classes de problemas.

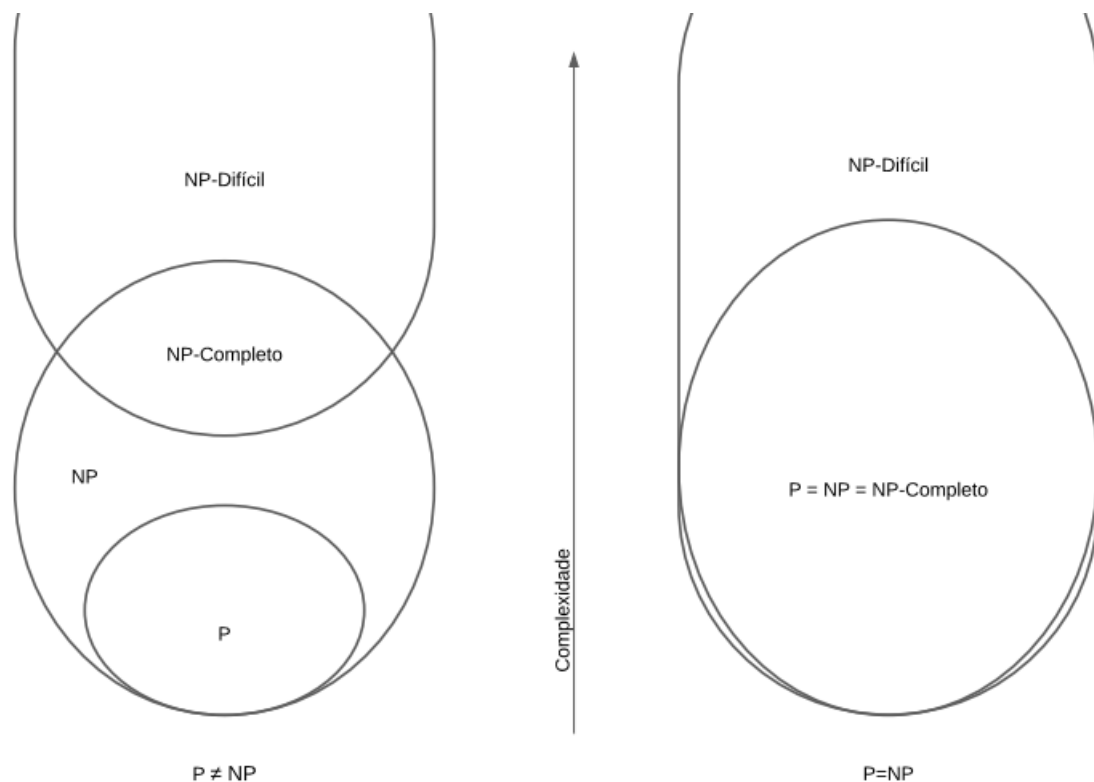


Figura 2.1: Diagrama de Euler para P, NP, NP-Completo e NP-Difícil Fonte: Libório, Felipe [1]

Entre os métodos, de otimização existentes algumas possíveis abordagens para resolução desse problema podem ser resumidas em duas classes: métodos exatos e heurísticos. Onde métodos exatos têm como objetivo buscar uma solução ótima para o problema e os métodos heurísticos buscam soluções próximas do ótimo em tempo computacional viável.

3 *Algoritmos Exatos e Meta-Heurísticos*

3.1 Algoritmos Exatos

De acordo com Carvalho, (2011)[4], os métodos de busca por soluções denominados exatos refere-se ao procedimento que encontra a solução ótima para o problema, porém o uso desses métodos para problemas com tal complexidade pode ser inviável. Nos métodos exatos são consideradas principalmente técnicas do tipo *relax-and-cut* e *branch-and-bound* descritos mais adiante.

De acordo com Libório(2019)[1], alguns problemas, de tempo polinomial determinístico, podem ser solucionados em tempo viável com o uso de algoritmos exatos. Já em problemas de tempo polinomial não-determinístico, não há solução ótima de tempo polinomial conhecida. Assim, para instâncias grandes destes problemas, busca-se usar técnicas que gerem soluções aproximadas de boa qualidade e que possam ser executadas em tempo hábil.

Por exemplo: suponha que um algoritmo $O(2^n)$ possa resolver um problema com 40 cursos em 30 horas em um computador que realiza 10^7 operações desse algoritmo por segundo. Então, levaria cerca de 60 horas para 41 cursos, e quase 2 anos para 49 cursos, mesmo com pequenas alterações no tamanho da entrada, o tempo de resposta aumenta exponencialmente. Por esse motivo segundo Carvalho, (2011).[4] métodos heurísticos são geralmente utilizados nesse tipo de problema.

3.1.1 Branch and Bound

De acordo com Rust[8], desde 1960 quando Land e Doig apresentaram um algoritmo computacional denominado "Branch and Bound", para a resolução de problemas de programação inteira mista, uma grande variedade de algoritmos baseados na técnica foram desenvolvidos e aperfeiçoados em diversas aplicações.

Segundo Filho e Lopes [9] por se tratar de um método enumerativo, a solução é obtida

através do particionamento do espaço de soluções, ou seja, o problema original é repetidamente decomposto em subproblemas menores até que uma solução seja obtida. O método aplicado permite identificar tal solução como a melhor possível ou ótima dentre todas as soluções viáveis para o problema. Para tornar isso possível, são construídas, dinamicamente, estruturas de árvore, de forma a representar todos os possíveis subproblemas, derivados a partir do problema original. Assim, o problema original é localizado na raiz da árvore e dá origem a todos os demais por decomposição.

3.1.2 Relax and Cut

De acordo com Pereira [10], o termo "Relax and Cut" foi originalmente utilizado por Escudero et al.[11] para denominar unicamente o algoritmo de ABOUDI et al.[12]. Segundo Cavalcante [13], o termo relax-and-cut tem sido usualmente utilizado na literatura para denotar uma larga classe de algoritmos baseados em relaxação Lagrangiana, onde desigualdades são dualizadas somente quando violadas pela solução corrente do Problema Lagrangiano Relaxado, que é uma técnica utilizada em otimização para simplificar problemas que possuem restrições difíceis de manejar.

3.2 Algoritmos Meta-Heurísticos

Segundo Luzia e Rodrigues [14] as metaheurísticas estão inclusas nos métodos de otimização estocástica, que por sua vez estuda os casos em que as restrições ou os parâmetros dependem de variáveis aleatórias.

De acordo com a definição original, metaheurísticas são métodos de solução que coordenam procedimentos de busca locais com estratégias de mais alto nível, de modo a criar um processo capaz de escapar de mínimos locais e realizar uma busca robusta no espaço de soluções de um problema.

3.2.1 Busca Local

Algoritmos de busca local são construídos como uma forma de exploração do espaço de busca. Nestes, um estado inicial é gerado por algum método (construtivo ou aleatório) e este é melhorado a cada iteração até que um estado ótimo seja atingido. De acordo com Gomes[15] diferente do que pode vir a ocorrer com outros procedimentos, a Busca Tabu(BT) não é confundida pela ausência de vizinhos aprimorantes. Isso significa que o método retorna a um ótimo

local visitado previamente, de forma a superar a otimalidade e atingir um resultado ótimo ou próximo ao ótimo global. A Figura 3.1 apresenta um algoritmo básico da BT.

Algorithm 1 Busca Tabu

```

1: Entrada:  $f(\cdot), N(\cdot), A(\cdot), |V|, |T|, s$ 
2: Saída:  $s^*$ 
3:  $s^* \leftarrow s$            {Melhor solução encontrada até então}
4:  $Iter \leftarrow 0$        {Contador do número de iterações}
5:  $MelhorIter \leftarrow 0$  {Iteração mais recente que forneceu  $s^*$ }
6:  $T \leftarrow \emptyset$    {Lista Tabu}
7: Inicialize a função de aspiração  $A$ 
8: while Critério de parada não satisfeito do
9:    $Iter \leftarrow Iter + 1$ 
10:  Seja  $s' \leftarrow s \oplus m$  o melhor elemento de  $V \subseteq N(s)$  tal que o movimento  $m$  não
    seja tabu ( $m \notin T$ ) ou  $s'$  atenda a condição de aspiração ( $f(s') < A(f(s))$ )
11:  Atualize a lista tabu  $T$ 
12:   $s \leftarrow s'$ 
13:  if  $f(s) < f(s^*)$  then
14:     $s^* \leftarrow s$ 
15:     $MelhorIter \leftarrow Iter$ 
16:  end if
17:  Atualize a função de aspiração  $A$ 
18: end while
19: Retorne  $s^*$ 

```

Figura 3.1: Algoritmo básico da Busca Tabu apresentado por Souza[16]

A BT utiliza estruturas de memória flexíveis para armazenar o conhecimento referente aos espaços que já foram percorridos. Por conta de utilizar tais estruturas, a técnica consegue proporcionar “boas” soluções, onde “boas” denota soluções geralmente perto do ótimo global em um tempo de processamento aceitável.

As Meta-heurísticas são desenvolvidas para fugirem destes ótimos locais e convergirem para uma solução ótima global.

De forma generalizada, uma técnica de busca local, começando de uma solução inicial s_0 (a qual pode ser gerada de forma aleatória ou obtida através do emprego de alguma outra técnica), navega pelo espaço de pesquisa, passando, iterativamente, de uma solução para outra que seja sua vizinha.

3.2.2 Algoritmo Genético

De acordo com Miranda [17] os Algoritmos Genéticos (AG) são modelos computacionais inspirados no princípio da evolução, proposto por C. Darwin que incorporam uma solução potencial para um problema específico numa estrutura semelhante a de um cromossomo e aplicam operadores de seleção e *crossover* a essas estruturas de forma a preservar informações críticas relativas à solução do problema.

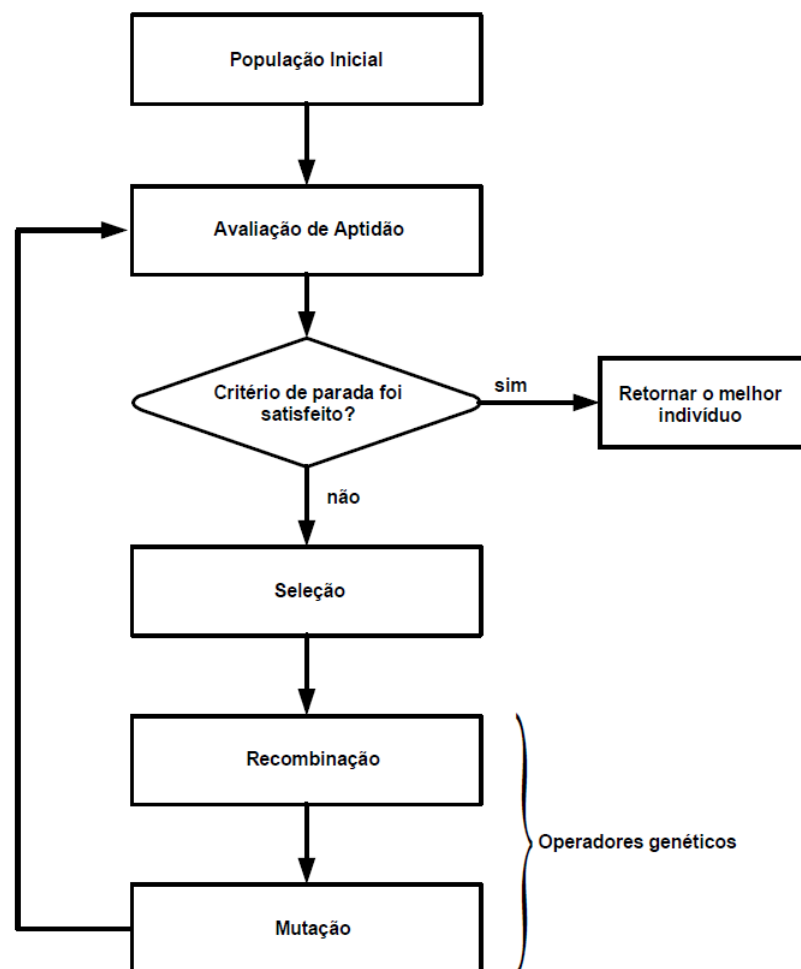


Figura 3.2: Estrutura básica de um Algoritmo Genético simples. Fonte: Miranda [17]

Na Figura 3.2 observa-se que cada iteração do algoritmo genético corresponde à aplicação de um conjunto de quatro operações básicas: cálculo de aptidão, seleção, cruzamento e mutação, onde:

- **População Inicial:** Uma população de n indivíduos é gerada aleatoriamente em que cada um dos indivíduos da população representa uma possível solução para o problema.
- **Avaliação de Aptidão:** Geralmente a aptidão do indivíduo é determinada através do cál-

culo da função objetivo, que depende das especificações de projeto.

- **Seleção:** Proporcionalmente com sua aptidão os indivíduos da geração atual são selecionados. Esses indivíduos são utilizados para gerar uma nova população por cruzamento.
- **Reprodução(*Crossover*):** Os indivíduos selecionados na etapa anterior são cruzados da seguinte forma: a lista de indivíduos selecionados é embaralhada aleatoriamente criando-se, desta forma, uma segunda lista, chamada lista de parceiros. Cada indivíduo selecionado é então cruzado com o indivíduo que ocupa a mesma posição na lista de parceiros.
- **Mutação:** A operação de mutação é utilizada para garantir uma maior varredura do espaço de estados e evitar que o algoritmo genético convirja muito cedo para mínimos locais. A mutação é efetuada alterando-se o valor de um gene de um indivíduo sorteado aleatoriamente com uma determinada probabilidade, denominada probabilidade de mutação, ou seja, vários indivíduos da nova população podem ter um de seus genes alterado aleatoriamente.

A forma como é feito o *crossover* é ilustrada na Figura 3.3 Os cromossomos de cada par de indivíduos a serem cruzados são particionados em um ponto, chamado ponto de corte, sorteado aleatoriamente. Um novo cromossomo é gerado permutando-se a metade inicial de um cromossomo com a metade final do outro.

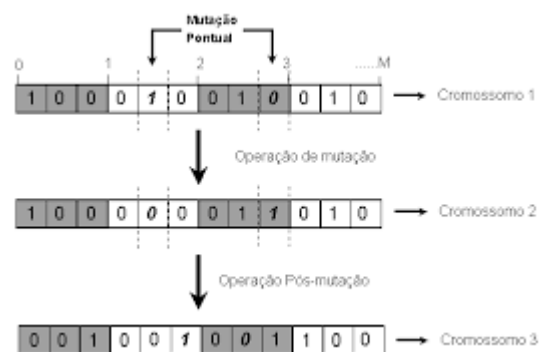


Figura 3.3: *Crossover* de indivíduos em um AG simples. Fonte: RODRIGUES, F.L. et al.[18]

Cada cromossomo de um AG representa uma solução do problema, tomado também como sinônimo de indivíduo, representado por uma cadeia de símbolos (genes). Uma das principais formas de representar um cromossomo é através de uma sequência de bits, onde cada bit ativo (1) representa a presença de uma característica e cada bit inativo (0) representa a ausência dessa característica.

3.2.3 GRASP(*Greedy Randomized Adaptive Search Procedure*)

De acordo com Resende e Ribeiro[22], GRASP é uma metaheurística *multi start* para problemas combinatórios, em que cada iteração consiste basicamente em duas fases: construção e busca local. A fase de construção constrói uma solução viável, cuja vizinhança é investigada até que um mínimo local seja encontrado durante a fase de busca local. As soluções iniciais são geradas por algum tipo de construção randômica gulosa ou algum esquema de perturbação. A melhor solução geral é mantida como resultado.

O algoritmo na figura 3.4 apresenta a construção das soluções.

```
Enquanto condiçãoTerminoNãoEncontrada faça  
    s ← construçãoSolução();  
    s' ← aplicaBuscaLocal(s);  
    memorizaMelhorSoluçãoEncontrada()  
fim enquanto
```

Figura 3.4: Algoritmo básico GRASP. Fonte: Fredo e Brito[23]

De acordo com Fredo e Brito[23], na fase de construção uma solução é construída elemento a elemento. Para a seleção de um elemento para a solução parcial, a princípio todos os candidatos são considerados, contudo o número de candidatos pode em muitos casos ser extremamente elevado. Por isso, normalmente, é considerado apenas uma lista restrita de candidatos (LRC), lista esta que possui os melhores elementos da lista de candidatos (LC). A LRC é composta por elementos p que atendam a Equação 3.1

$$CardinalidadeLRC = CardinalidadeLC * \alpha \quad (3.1)$$

Pela forma como são construídas as soluções na etapa inicial, essas não representam, na maioria dos casos, um ótimo local, por esse motivo é fortemente recomendada a etapa de busca local. As iterações do GRASP são totalmente independentes, esse é o motivo de ser também conhecida como uma heurística do tipo *multi start*.

A segunda fase do GRASP ou fase de busca local consiste em refinar a solução encontrada na primeira fase aplicando um método de busca local. Isso corresponde a uma intensificação na solução encontrada explorando regiões vizinhas através da busca local para encontrar um ótimo local.

A busca local pode ser sofisticada, entretanto não se deve esquecer do diferencial do GRASP que é amostrar o espaço com gerações rápidas. Quanto melhor for a qualidade da solução gerada na primeira fase, maior será a velocidade para encontrar um ótimo local pela fase de busca local.

Os algoritmos a serem utilizados nesta segunda fase podem ser básicos como o Hill-Climbing ou metaheurísticas mais avançadas como Tabu, SA e VND.

É importante salientar que o GRASP não faz uso de históricos no processo de busca. É possível armazenar uma ou várias soluções sendo as melhores até o momento. Entretanto, o GRASP é simples, rápido e pode ser facilmente integrado com outras técnicas de busca.[23]

4 *Definição do Problema e Proposta para solução do problema*

A Equação 4.1, proposta por Frangouli et al. (1995)[19], é utilizada para determinar o espaço de pesquisa máximo para problemas de quadro de horário de cursos universitários e se mostra eficaz para o universo explorado neste estudo. Sendo B o tamanho do espaço de pesquisa, b_d o número de dias letivos na semana, b_p o número de períodos disponíveis para a programação das aulas, b_s o número de salas disponíveis, b_m e b_d como sendo o número médio de aulas para cada disciplina, e o número de disciplinas respectivamente, temos:

$$B = (b_d \times b_p \times b_s)^{b_m \times b_d} \quad (4.1)$$

Considerando um semestre com a atual grade de disciplinas do curso de Tecnologia da Informação e Comunicação, onde aulas das 31 disciplinas devem ser alocadas, em média cada disciplina possui 3,9 tempos de aulas semanais, distribuídas em 6 dias letivos na semana, com 7 períodos (horários) de aulas, e que há 5 salas para que uma aula seja atribuída. Nesta suposição o espaço de busca a ser examinado para a construção de um calendário semanal é igual a $B = (6 \times 7 \times 5)^{3,9 \times 31}$, ou seja, 210^{121} .

Sendo um problema de análise combinatória, a criação de uma grade de horários se torna um processo difícil, já que existem milhares de combinações possíveis, quando levamos em consideração somente a quantidade de total de horários por matéria e o quadro de professores. A criação da grade de horários, se torna ainda mais difícil considerando que se deve escolher a melhor opção. Esta escolha, quando a confecção é feita manualmente, pode ser feita via tentativa e erro, porém esta opção requer ainda mais tempo.

Com isso foi identificada a oportunidade de automatizar esse processo, criando um sistema que gere a grade de horário da faculdade, levando em conta as 31 disciplinas do curso, distribuídas em 5 Períodos, onde estas disciplinas estão distribuídas de segunda a sábado, das 7h00min às 12h30min divididas entre 13 professores. A elaboração da grade deve seguir algumas regras, tais como:

- Disciplina não pode ser ministrada por mais de um professor;
- Disciplinas de um mesmo período não podem colidir horário entre elas;
- Professor não pode ministrar mais de uma disciplina no mesmo horário;
- Professores só podem ministrar suas aulas em horários previamente cadastrados.

Após análise das metodologias apresentadas percebeu-se que a metodologia GRASP oferece uma abordagem satisfatória para a solução do problema montagem de Quadro de Horários. A escolha desta técnica deve-se ao fato que tal metaheurística obteve bons resultados quando aplicada a problemas de dificuldade semelhante.

Na fase de construção GRASP (*Greedy Randomized Adaptive Search Procedure*), o primeiro passo é buscar as matérias cadastradas e cria-se uma lista de matérias M . Busca-se uma matéria aleatória da lista de matérias, e para M_i busca-se atribuir um Professor p , também aleatório, de uma lista de Professores ligados a M_i gerando uma turma T para cada professor P_j . Criando assim, uma lista de candidatos (LC). Por último seleciona-se uma disponibilidade d do P_j aleatoriamente e tenta fazer a atribuição na lista de turmas da solução s final, o processo de montagem da LC pode ser visualizado no pseudocódigo abaixo 4.1

```

M = BuscaMatérias( );
Para i = 0 até |M| faça
    m = M[random];
    P = ObterProfessoresDaMateria(m.MaterialId);
    Para j = 0 até |P| faça
        Turma turma = new Turma( ){
            Professor = P[j];
            Matéria = m;
        };
        LC.Add(turma);
    fim_para
fim_para

```

Figura 4.1: Montagem da Lista de Candidatos. Fonte: Ferreira, João Pedro Moura

Na fase da busca local, verifica-se se a solução gerada é uma solução válida e caso sim verifica-se o tempo, considerando como melhor solução a solução gerada em menor tempo.

Todo este processo é realizado repetidamente x quantidade de iterações definida inicialmente que pode ser representado pelo código abaixo 4.2:

```
public static void Executar(Grade grade)
{
    var inicio = DateTime.Now;
    var gradeController = new GradeController();
    grade.GradeId = gradeController.IncluirGrade(grade);

    var materiaController = new MateriaController();
    var qtdAulas = materiaController.ObterQtdAulasSemanais();

    var melhorSolucao = new Solucao();
    melhorSolucao = melhorSolucao.GerarSolucao(grade.GradeId);
    while (melhorSolucao.Turmas.Count < qtdAulas)
    {
        melhorSolucao = melhorSolucao.GerarSolucao(grade.GradeId);
    }

    var melhorCusto = melhorSolucao.Custo;

    int iteracoes = 0;
    while (iteracoes < grade.QtdIteracoes)
    {
        var solucao = new Solucao();
        solucao = solucao.GerarSolucao(grade.GradeId);
        var qtdAulasSemana = solucao.Turmas.Count;

        var qtdMateriaAdd = solucao.VerificarQtdMateriaAdd(solucao.Turmas);
        solucao.Sucesso = solucao.VerificarSolucaoValida(qtdMateriaAdd, qtdAulasSemana);
        if (solucao.Sucesso)
        {
            if (solucao.Custo < melhorCusto)
            {
                melhorSolucao = solucao;
                melhorCusto = solucao.Custo;
            }
        }
        iteracoes++;
    }

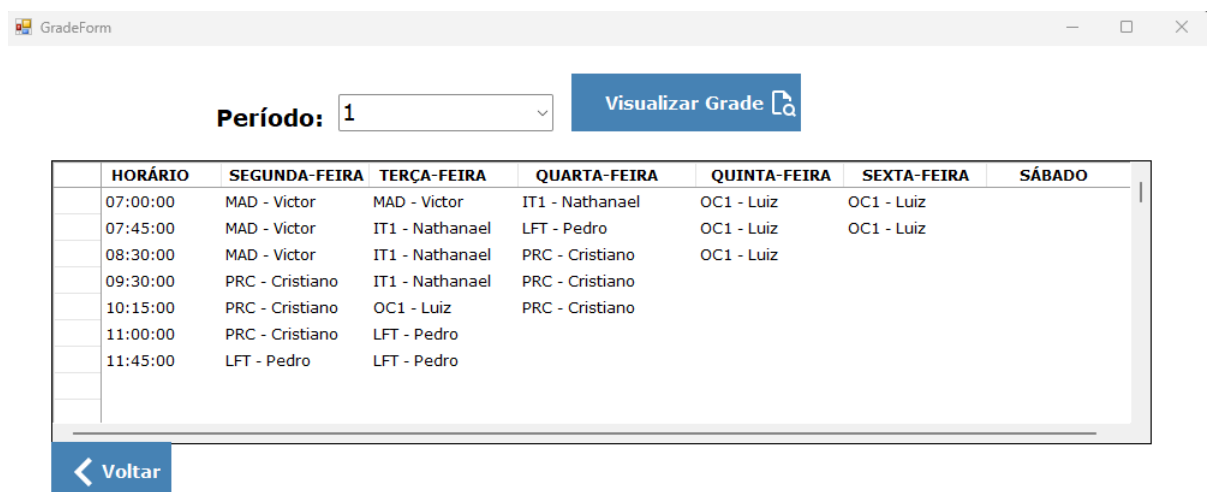
    var fim = DateTime.Now;
    var custo = fim - inicio;
    grade.TempoCriacao = custo;
    gradeController = new GradeController();
    gradeController.AtualizarTempo(grade);
}
```

Figura 4.2: Código Final. Fonte: Ferreira, João Pedro Moura

5 Solução

5.1 Resultados

A seguir, são apresentadas as grades de horário geradas a partir da solução desenvolvida no presente trabalho, demonstrando os resultados obtidos através do programa implementado.



The screenshot shows a web application window titled "GradeForm". It features a dropdown menu for "Período:" set to "1" and a blue button labeled "Visualizar Grade" with a magnifying glass icon. Below this is a table with 7 columns: "HORÁRIO", "SEGUNDA-FEIRA", "TERÇA-FEIRA", "QUARTA-FEIRA", "QUINTA-FEIRA", "SEXTA-FEIRA", and "SÁBADO". The table contains 6 rows of data. At the bottom left of the table area is a blue button labeled "Voltar" with a left arrow icon.

HORÁRIO	SEGUNDA-FEIRA	TERÇA-FEIRA	QUARTA-FEIRA	QUINTA-FEIRA	SEXTA-FEIRA	SÁBADO
07:00:00	MAD - Victor	MAD - Victor	IT1 - Nathanael	OC1 - Luiz	OC1 - Luiz	
07:45:00	MAD - Victor	IT1 - Nathanael	LFT - Pedro	OC1 - Luiz	OC1 - Luiz	
08:30:00	MAD - Victor	IT1 - Nathanael	PRC - Cristiano	OC1 - Luiz		
09:30:00	PRC - Cristiano	IT1 - Nathanael	PRC - Cristiano			
10:15:00	PRC - Cristiano	OC1 - Luiz	PRC - Cristiano			
11:00:00	PRC - Cristiano	LFT - Pedro				
11:45:00	LFT - Pedro	LFT - Pedro				

Figura 5.1: Grade de Horário do Primeiro Período. Fonte: Ferreira, João Pedro Moura

GradeForm

Período: 2 Visualizar Grade

HORÁRIO	SEGUNDA-FEIRA	TERÇA-FEIRA	QUARTA-FEIRA	QUINTA-FEIRA	SEXTA-FEIRA	SÁBADO
07:00:00	POO - Cristiano	POO - Cristiano	BD1 - Sandra		MAT - Victor	POI - Nathanael
07:45:00	POO - Cristiano	POO - Cristiano	BD1 - Sandra		MAT - Victor	
08:30:00	POO - Cristiano	POO - Cristiano	BD1 - Sandra		MAT - Victor	
09:30:00	OC2 - Thales	OC2 - Thales	IT2 - Nathanael			
10:15:00	OC2 - Thales	OC2 - Thales	IT2 - Nathanael			
11:00:00	OC2 - Thales	OC2 - Thales	IT2 - Nathanael			
11:45:00	MAT - Victor	MAT - Victor	POI - Nathanael			

Voltar

Figura 5.2: Grade de Horário do Segundo Período. Fonte: Ferreira, João Pedro Moura

GradeForm

Período: 3 Visualizar Grade

HORÁRIO	SEGUNDA-FEIRA	TERÇA-FEIRA	QUARTA-FEIRA	QUINTA-FEIRA	SEXTA-FEIRA	SÁBADO
07:00:00	POO - Cristiano	POO - Cristiano	BD1 - Sandra		MAT - Victor	POI - Nathanael
07:45:00	POO - Cristiano	POO - Cristiano	BD1 - Sandra		MAT - Victor	
08:30:00	POO - Cristiano	POO - Cristiano	BD1 - Sandra		MAT - Victor	
09:30:00	OC2 - Thales	OC2 - Thales	IT2 - Nathanael			
10:15:00	OC2 - Thales	OC2 - Thales	IT2 - Nathanael			
11:00:00	OC2 - Thales	OC2 - Thales	IT2 - Nathanael			
11:45:00	MAT - Victor	MAT - Victor	POI - Nathanael			

Voltar

Figura 5.3: Grade de Horário do Terceiro Período. Fonte: Ferreira, João Pedro Moura

GradeForm

Período: 4 Visualizar Grade

HORÁRIO	SEGUNDA-FEIRA	TERÇA-FEIRA	QUARTA-FEIRA	QUINTA-FEIRA	SEXTA-FEIRA	SÁBADO
07:00:00	SO2 - Thales	SO2 - Thales	SIN - José Carlos	SIN - José Carlos	PAV - Gustavo	
07:45:00	SO2 - Thales	TCD - Pedro	SIN - José Carlos		RD2 - Isadora	
08:30:00	SO2 - Thales	TCD - Pedro	SIN - José Carlos		PAV - Gustavo	
09:30:00	ICG - Marcos	ICG - Marcos	TCD - Pedro		PAV - Gustavo	
10:15:00	ICG - Marcos	ICG - Marcos	TCD - Pedro			
11:00:00	ICG - Marcos		RD2 - Isadora			
11:45:00	PAV - Gustavo	RD2 - Isadora	RD2 - Isadora			

Voltar

Figura 5.4: Grade de Horário do Quarto Período. Fonte: Ferreira, João Pedro Moura

GradeForm

Período: 5

Visualizar Grade

HORÁRIO	SEGUNDA-FEIRA	TERÇA-FEIRA	QUARTA-FEIRA	QUINTA-FEIRA	SEXTA-FEIRA	SÁBADO
07:00:00	EMP - Marissa	SAS - Luiz		PAI - Isaque		
07:45:00	EMP - Marissa	MQA - Victor		PAI - Isaque		
08:30:00	PEA - Marissa	SAS - Luiz		SMH - Isaque		
09:30:00	MQA - Victor	SAS - Luiz	SAD - José Carlos	SAS - Luiz		
10:15:00	MQA - Victor	CEL - Marissa	SAD - José Carlos	SMH - Isaque		
11:00:00	MQA - Victor	CEL - Marissa		LEI - Isaque		
11:45:00	PEA - Marissa			LEI - Isaque		

<

Voltar

Figura 5.5: Grade de Horário do Quinto Período. Fonte: Ferreira, João Pedro Moura

5.2 Caso de Uso

5.2.1 Diagrama de Caso de Uso

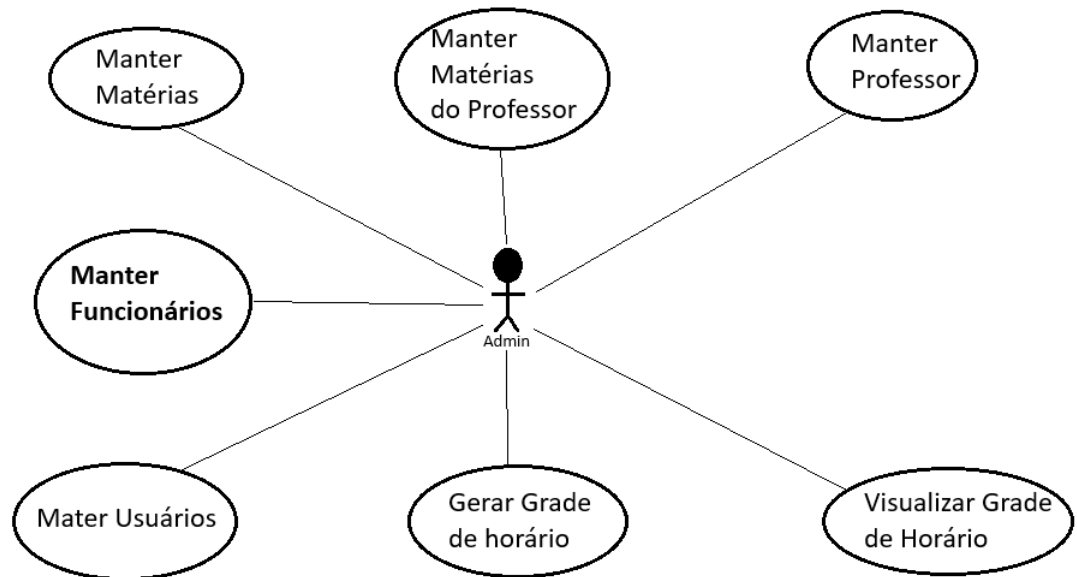


Figura 5.6: Diagrama de Caso de Uso. Fonte: Ferreira, João Pedro Moura

5.2.2 Especificações dos Casos de Uso

Caso de Uso 1 Manter Professores

Atores

Admin

Roteiro

- O Admin, através da área de administração principal, acessa a área de administração de professores.
- O sistema redireciona para a tela com a lista de Professores cadastrados.
- O Admin seleciona um professor;
- O Admin escolhe uma das opções possíveis:

- Adicionar
- Editar
- Excluir
- Exibir Detalhes
- Pesquisar

Fluxos Alternativos

FA01 Adicionar

- O Admin seleciona a opção "Adicionar" e o sistema abre a página de cadastro de professores
- O Admin preenche as informações necessárias
- O Admin salva as informações
- O Sistema valida as informações
- O Sistema salva o novo Professor
- O Sistema grava a ação em Log
- O sistema redireciona para a tela com a lista de professores cadastrados.

FA02 Editar

- O Admin seleciona a opção "Editar" e o sistema abre a página de edição de professores
- O Admin preenche as informações necessárias;
- O Admin salva as informações
- O Sistema valida as informações
- O Sistema edita o Professor selecionado;
- O Sistema grava a ação em Log
- O sistema redireciona para a tela com a lista de professores cadastrados.

FA03 Excluir

- O Admin seleciona a opção "Excluir"
- O Sistema verifica se o professor está associado à uma matéria.
 - Caso esteja exibindo a mensagem dizendo que não pode realizar a exclusão, e grava a tentativa em Log
 - Caso não, faz a exclusão dos dados do sistema fazendo a gravação em Log.
- O sistema redireciona para a tela com a lista de professores cadastrados.

FA04 Exibir Detalhes

- O Admin seleciona a opção "Exibir Detalhes" e o sistema abre a página com os detalhes do professor.

FA05 Pesquisar

- O Admin preenche as informações necessárias
- O Admin seleciona a opção "Pesquisar"
- O Sistema valida as informações
- O Sistema lista o resultado da pesquisa

Caso de Uso 2 Manter Matérias

Atores

Admin

Roteiro

- O Admin, através da tela inicial, acessa a área de administração de matérias
- O sistema redireciona para a tela com a lista de Matérias cadastradas
- O Admin seleciona uma Matéria
- O Admin escolhe uma das opções possíveis
 - Adicionar

- Excluir
- Exibir Detalhes
- Pesquisar

Fluxos Alternativos

FA01 Adicionar

- O Admin seleciona a opção "Adicionar" e o sistema abre a página de cadastro de matérias
- O Admin preenche as informações necessárias;
- O Admin salva as informações
- O Sistema valida as informações
- O Sistema salva a nova Matéria
- O Sistema grava a ação em Log
- O sistema redireciona para a tela com a lista de matéria cadastradas

FA02 Excluir

- O Admin seleciona a opção "Excluir"
- O Sistema verifica se a matéria está associado à um Professor
 - Caso esteja exibida a mensagem dizendo que não pode realizar a exclusão, e grava a tentativa em Log
 - Caso não, faz a exclusão dos dados do sistema fazendo a gravação em Log
- O sistema redireciona para a tela com a lista de matérias cadastradas

FA03 Exibir Detalhes

- O Admin seleciona a opção "Exibir Detalhes" e o sistema abre a página com os detalhes da matéria

FA04 Pesquisar

- O Admin preenche as informações necessárias

- O Admin seleciona a opção "Pesquisar"
- O Sistema valida as informações
- O Sistema lista o resultado da pesquisa

Caso de Uso 3 Manter Matérias do Professor

Atores

Admin

Roteiro

- O Admin, através da tela inicial, acessa a área de administração de Matérias do Professor
- O sistema redireciona para a tela com a lista de Matérias dos Professores cadastradas
- O Admin seleciona uma ligação entre Matéria e Professor.
- O Admin escolhe uma das opções possíveis:
 - Adicionar
 - Excluir
 - Exibir Detalhes
 - Pesquisar

Fluxos Alternativos

FA01 Adicionar

- O Admin seleciona a opção "Adicionar" e o sistema abre a página de cadastro de matérias dos professores
- O Admin preenche as informações necessárias
- O Admin salva as informações
- O Sistema valida as informações
- O Sistema salva a nova Matéria do Professor
- O Sistema grava a ação em Log

- O sistema redireciona para a tela com a lista de Matérias dos Professores cadastradas

FA03 Excluir

- O Admin seleciona a opção "Excluir"
- O Sistema exclui a ligação entre professor e matéria
- O sistema redireciona para a tela com a lista de matérias dos professores cadastradas

FA04 Exibir Detalhes

- O Admin seleciona a opção "Exibir Detalhes" e o sistema abre a página com os detalhes da Matéria do Professor

FA04 Pesquisar

- O Admin preenche as informações necessárias
- O Admin seleciona a opção "Pesquisar"
- O Sistema valida as informações
- O Sistema lista o resultado da pesquisa

Caso de Uso 4 Manter Funcionários

Atores

Admin

Roteiro

- O Admin, através da tela inicial, acessa a área de administração de Funcionários
- O sistema redireciona para a tela com a lista de Funcionários
- O Admin seleciona um Funcionário
- O Admin escolhe uma das opções possíveis
 - Adicionar

- Excluir
- Editar
- Exibir Detalhes
- Pesquisar

Fluxos Alternativos

FA01 Adicionar

- O Admin seleciona a opção "Adicionar" e o sistema abre a página de cadastro de Funcionários
- O Admin preenche as informações necessárias
- O Admin salva as informações
- O Sistema valida as informações
- O Sistema salva o Funcionário
- O Sistema grava a ação em Log
- O sistema redireciona para a tela com a lista de Funcionários

FA02 Excluir

- O Admin seleciona a opção "Excluir"
- O Sistema exclui o Usuário associado ao aluno
- O Sistema grava a ação em Log
- O Sistema exclui o Funcionário
- O Sistema grava a ação em Log
- O sistema redireciona para a tela com a lista de Funcionários

FA02 Editar

- O Admin seleciona a opção "Editar" e o sistema abre a página de Edição de Funcionários
- O Admin preenche as informações necessárias.

- O Admin salva as informações
- O Sistema edita o Usuário associado ao aluno.
- O Sistema grava a ação em Log
- O Sistema edita o Funcionário
- O Sistema grava a ação em Log
- O sistema redireciona para a tela com a lista de Funcionários

FA03 Exibir Detalhes

- O Admin seleciona a opção "Exibir Detalhes" e o sistema abre a página com os detalhes do Funcionário

FA04 Pesquisar

- O Admin preenche as informações necessárias
- O Admin seleciona a opção "Pesquisar"
- O Sistema valida as informações
- O Sistema lista o resultado da pesquisa

Caso de Uso 6 Manter Usuários

Atores

Admin

Roteiro

- O Admin, através da tela inicial, acessa a área de administração de Usuários
- O sistema redireciona para a tela com a lista de Usuários
- O Admin seleciona um Usuário
- O Admin escolhe uma das opções possíveis:
 - Exibir Detalhes

- Pesquisar

Fluxos Alternativos

FA01 Exibir Detalhes

- O Admin seleciona a opção "Exibir Detalhes" e o sistema abre a página com os detalhes do Funcionário

FA02 Pesquisar

- O Admin preenche as informações necessárias
- O Admin seleciona a opção "Pesquisar"
- O Sistema valida as informações
- O Sistema lista o resultado da pesquisa

Caso de Uso 7 Manter Grade de Horário

Atores

Admin

Roteiro

- O Admin, através da tela inicial, seleciona a opção de Grade de Horários
- O Sistema verifica se já existe uma grade de horários cadastrada
 - Caso não, o Sistema realiza a gravação de uma nova grade, e gera a gravação da ação em Log
 - Caso sim, o Sistema apresenta a pergunta se deseja realizar o cadastro de uma nova grade
 - * Caso não, exibe a tela da Grade De Horários com a grade Atual
 - O Funcionário escolhe qual o Período desejado
 - O Sistema exibe a grade do Período desejado
 - * Caso sim, o Sistema realiza a gravação de uma nova grade, a opção sim gera a gravação da ação em Log

5.3 Descrição das Classes

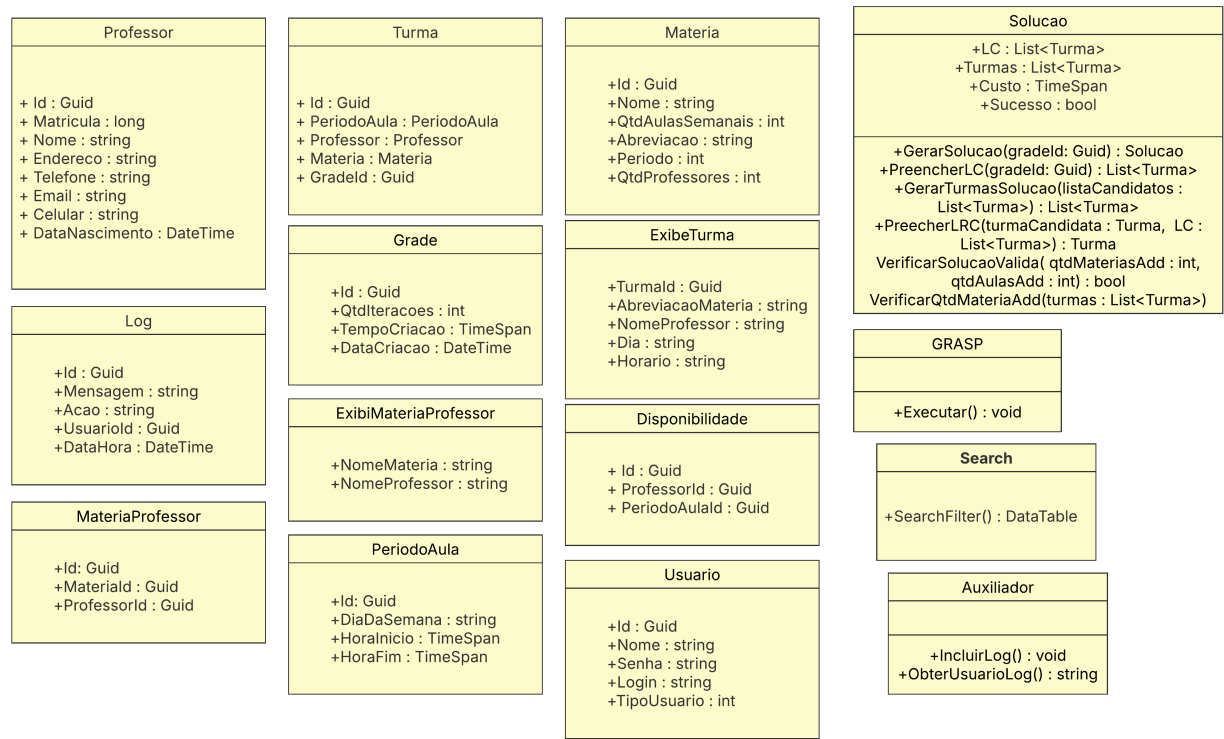


Figura 5.7: Diagrama de Classes. Fonte: Ferreira, João Pedro Moura

5.4 Diagrama de Entidade e Relacionamentos.

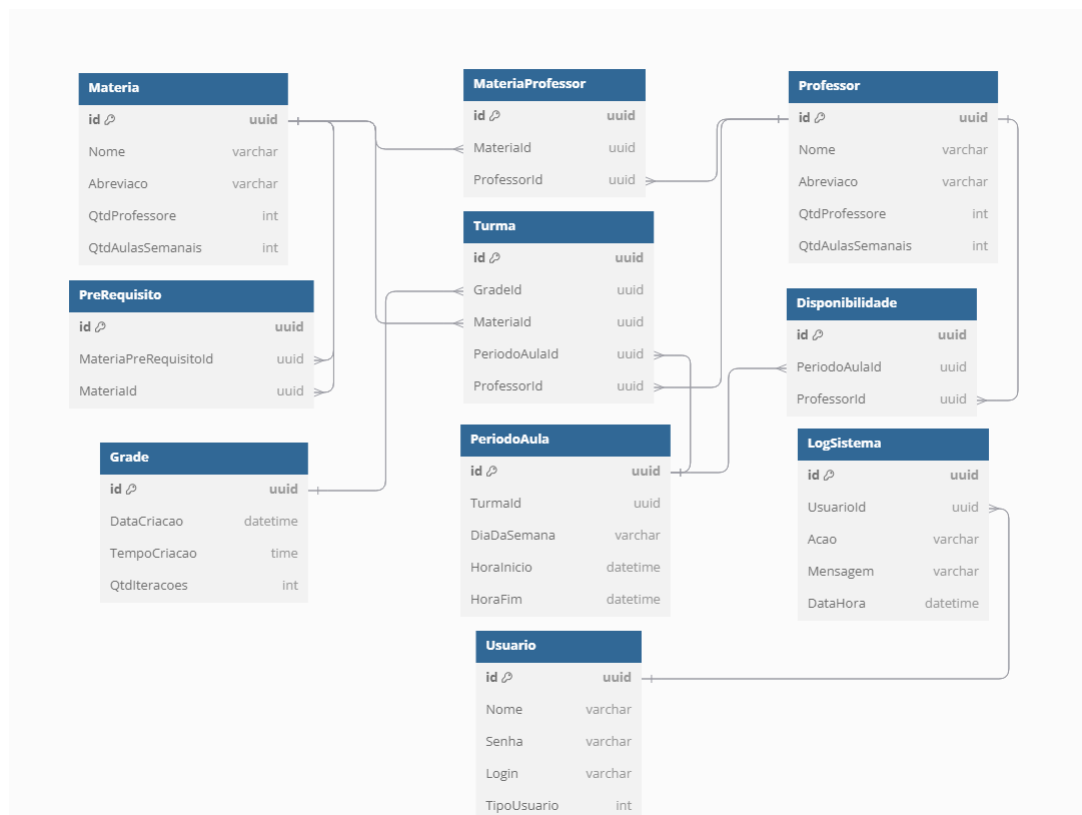


Figura 5.8: Diagrama de Entidade e Relacionamento. Fonte: Ferreira, João Pedro Moura

5.5 Tecnologias Utilizadas

5.5.1 Windows Forms .NET

Windows Forms é uma estrutura de interface do usuário para criar Windows aplicativos de área de trabalho. Ele fornece uma das maneiras mais produtivas de criar aplicativos de área de trabalho com base no designer visual fornecido no Visual Studio. Funcionalidades como o posicionamento do tipo "arrastar e soltar" de controles visuais facilitam a criação de aplicativos da área de trabalho.

com o Windows Forms, você desenvolve aplicativos graficamente ricos que são fáceis de implantar, atualizar e trabalhar enquanto estiverem offline ou conectados à internet. Windows Forms aplicativos podem acessar o hardware local e o sistema de arquivos do computador em que o aplicativo está sendo executado.

5.5.2 Visual C#

O C# é uma linguagem de programação criada para o desenvolvimento de uma variedade de aplicações que executam sobre o *.NET Framework*. C# é uma linguagem simples, poderosa, com tipagem segura e orientada a objetos. As várias inovações no C# permitem o desenvolvimento rápido de aplicações, mantendo a expressividade e a elegância das linguagens C-style. A biblioteca de classes do *.NET Framework* fornece acesso a vários serviços do sistema operacional e outras classes úteis e bem estruturadas que aceleram significativamente o ciclo de desenvolvimento.

5.5.3 Microsoft Visual Studio Community 2022

Visual Studio Community 2022 é um ambiente de desenvolvimento integrado completo, extensível e gratuito para a criação de aplicativos modernos para Android, iOS, Windows, Linux, bem como aplicativos da Web, serviços em nuvem e jogos. Código em C#, Visual Basic, F#, C++ e muito mais. Além de ferramentas integradas para projetar, editar, analisar, depurar, testar e implantar código em qualquer plataforma. O Visual Studio Community é gratuito para indivíduos e organizações envolvidas em treinamento e educação, pesquisa acadêmica, desenvolvimento de projetos de código aberto, drivers de dispositivo Windows e desenvolvimento de SQL.

5.5.4 Microsoft SQL Server 2017 Express

O Microsoft SQL Server 2017 Express um sistema gratuito de gerenciamento de dados avançado e confiável que fornece um repositório de dados confiável e avançado para sites leves e aplicativos de área de trabalho.

6 *Conclusão*

Este trabalho, explora como o problema da construção de Quadro de Horários é abordado pela área da computação, além de possíveis tratamentos para o ele, que por estar inserido na classe de problemas NP-Completo, a utilização de algoritmos exatos não é eficaz, e que o uso das meta-heurísticas ajudariam na solução com tempo hábil e de forma eficaz, Concluo que a utilização do algoritmo GRASP oferece uma boa solução para problemas deste tipo.

O objetivo deste trabalho foi concluído, com a obtenção de resultados satisfatórios a partir do sistema modelado.

As ferramentas utilizadas para a criação do sistema se mostraram eficazes, uma vez que suprem todas as necessidades para o desenvolvimento da solução e as referências utilizadas na pesquisa foram uma sólida base, além de agregar conhecimento sobre o assunto e a solução do problema proposto.

O algoritmo implementado se mostrou eficaz na montagem de uma solução válida, onde todas as disciplinas são ofertadas sem que exista colisões de horário, de professores e matérias do mesmo período. O tempo de execução do algoritmo varia de acordo com alguns fatores, tais como:

- Quantidade de Professores
- Quantidade de Horários Disponíveis para a inserção de uma Turma na solução
- Quantidade de Iterações

Pode se perceber que com um número de 120 iterações o sistema consegue chegar a uma solução que atenda todas as restrições mostradas anteriormente em um tempo de execução aceitável.

Ao fim deste projeto pode-se perceber que algumas funcionalidades poderiam agregar valor ao projeto, deixando o sistema mais robusto e trazendo opções como por exemplo uma área para professores e alunos com funcionalidades exclusivas para ambos perfis, como lançamento

de faltas e notas ou solicitações, tornando ele um sistema que pode ser utilizado por toda a unidade, e não somente para gestão de professores e criação da grade de horários.

A implementação da restrição de montagem por número de salas disponíveis, além da quantidade de alunos por turma, trariam, bons parâmetros para a montagem da solução.

A melhoria da Busca Local utilizada, seria um bom ponto de melhoria, seja mudando a forma como a mesma está sendo feita atualmente ou com a hibridação do método utilizado, utilizando outras metaheurísticas, como por exemplo a Busca Tabu, na fase de busca local, dessa forma, a solução gerada evitará possíveis inconsistências na grade gerada, como espaços sem aula e aulas da mesma disciplina separadas no mesmo dia.

Referências

- [1] LIBÓRIO, Felipe Tenório de Holanda Rocha. Algoritmos Exatos e Heurísticos para os Problemas de Steiner e de Conexão de Terminais com Número Restrito de Roteadores e Elos. 2019. 60 f. Trabalho de Conclusão de Curso (Bacharelado em Ciência da Computação) – Unidade Acadêmica de Garanhuns, Universidade Federal Rural de Pernambuco, Garanhuns, 2019.
- [2] Cooper, Tim B e Kingston, Jeffrey h. The Complexity of Timetable Construction Problems. Sydney-Aus. The University of Sydney. 1995
- [3] WREN, Anthony; Scheduling, Timetabling and Rostering - **A Special Relationship?, Practice and Theory of Automated Timetabling**. Springer, Berlin, Heidelberg, 1996.
- [4] CARVALHO, Rodrigo de. Abordagem Heurística para o Problema de Programação de Horários de Cursos. Belo Horizonte: Universidade Federal de Minas Gerais, 2011.
- [5] Bardadym, V.A., “**Computer-Aided School and University Timetabling: The New Wave**”, **Practice and Theory of Automated Timetabling**. Springer, Berlin, Heidelberg, 1996.
- [6] S. A. Cook, The complexity of theorem-proving procedures, in: STOC’71 Proceeding of the third annual ACM symposium on Theory of computing, 1971.
- [7] CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. **Introduction to algorithms. 2. ed.** Cambridge: MIT Press, 2002.
- [8] RUST, RONALDO Um Algoritmo Branch and Bound para resolução de Problemas de Localização Capacitados - (Rio de Janeiro) 1983
- [9] Filho, Devair de Oliveira e Lopes, Raul Henriques Cardoso ALGORITMOS BRANCH AND BOUND PARALELOS: RESULTADOS DE EXPERIMENTOS COMPUTACIONAIS, Universidade Federal do Espírito Santo, 2003
- [10] Pereira, Matheus Caminha Algoritmo Relax-and-Cut para o Problema do Conjunto Independente Máximo/Matheus Caminha Pereira. – Rio de Janeiro: UFRJ/COPPE, 2018.
- [11] L. Escudero, M. Guignard, and K. Malik. A Lagrangean relax and cut approach for the sequential ordering with precedence constraints. *Annals of Operations Research*, 50:219–237, 1994.
- [12] ABOUDI, R., HALLEFJORD, A., JORNSTEN, K. “A facet generation and relaxation technique applied to an assignment problem with side constraints”, *European Journal of Operational Research*, v. 3, n. 50, pp. 335–344, 1991.

- [13] Cavalcante, Victor Fernandes. Algoritmos relax-and-cut para problemas de programação inteira 0-1, 2008
- [14] Luzia, Leandro Ferro, Rodrigues Mauricio Chui . Introdução ao Escalonamento e Aplicações Estudo sobre as Metaheurísticas, Universidade de São Paulo, São Paulo. Disponível em: <https://www.ime.usp.br/~gold/cursos/2009/mac5758/LeandroMauricioHeuristica.pdf>
- [15] GOMES, André. Uma Introdução à Busca Tabu, Universidade de São Paulo, São Paulo. Disponível em: <https://www.ime.usp.br/~gold/cursos/2009/mac5758/AndreBuscaTabu.pdf>
- [16] Souza, Marcone J. F. e Penna, Puca H. V. Busca Tabu. Notas de aula de Técnicas Metaheurísticas para Otimização Combinatória. Departamento de Computação, Universidade Federal de Ouro Preto, Ouro Preto, 2021.
- [17] MIRANDA, Marcio de Nunes, Algoritmos Genéticos: Fundamentos e Aplicações, Disponível em: <http://www.nce.ufrj.br/GINAPE/VIDA/alggenet.htm>
- [18] Rodrigues, Flávio Lopes, Leite, Helio Garcia, Santos, Heleno do Nascimento, Souza, Agostinho Lopes de, Silva, Gilson Fernandes da. METAHEURÍSTICA ALGORITMO GENÉTICO PARA SOLUÇÃO DE PROBLEMAS DE PLANEJAMENTO FLORESTAL COM RESTRIÇÕES DE INTEGRIDADE, Universidade Federal de Viçosa, Minas Gerais, 2004. Disponível em: <https://www.scielo.br/j/rarv/a/GjchDFjXJj87v3bmMgVZdNC/?format=pdf&lang=pt>
- [19] FRANGOULI, Harikleia; HARMANDAS, Vassilis; STAMATOPOULOS, Panagiotis. Construction of optimum timetables for university course. Proc. of the Trird International Conference and Exhibition on Practical Applications of Prolog, 1995.
- [20] BATISTA, Victor Emanuel, Algoritmo Genético para o Problema da Mochila, 2020. disponível em: <https://bityli.com/algoritmo-genetico-para-o-problema-da-mochila>
- [21] Glover, Fred e Kochenberger, Gary A., **Handbook of Metaheuristics**, Kluwer. 2003
- [22] Resende C e Ribeiro C. "GRASP", em: Handbook of Metaheuristics. Kluwer. 2003
- [23] FREDO, Ademir Roberto e Brito, Robison Cris. Implementação da Metaheurística GRASP para o Problema do Caixeiro Viajante Simétrico, Universidade Federal do Paraná.
- [24] AN AUTOMATIC METHOD OF SOLVING DISCRETE PROGRAMMING PROBLEMS By A. H. LAND AND A. G. DOIG, 1960